```
class interrupt_flag
{
    std::atomic<bool> flag;
    std::condition_variable* thread_cond;
    std::condition_variable_any* thread_cond_any;
    std::mutex set_clear_mutex;

public:
    interrupt_flag():
    thread_cond(0),thread_cond_any(0)
{}

void set()
{
    flag.store(true,std::memory_order_relaxed);
    std::lock_guard<std::mutex> lk(set_clear_mutex);
    if(thread_cond)
    {
        thread_cond->notify_all();
    }
    else if(thread_cond_any)
    {
        thread_cond_any->notify_all();
    }
}

template<typename Lockable>
void wait(std::condition_variable_any& cv,Lockable& lk)
{
    struct custom_lock
    {
        interrupt_flag* self;
        Lockable& lk;

        custom_lock(interrupt_flag* self_,
                    std::condition_variable_any& cond,
                    Lockable& lk_):
            self(self_),lk(lk_)
        {
            self->set_clear_mutex.lock();    <-- (1)
            self->thread_cond_any=&cond;     <-- (2)
        }
```